

ExpertGym: Learning Agent Task-Vector Composition from Executable Feedback

Anonymous EMNLP Submission

Abstract

Task vectors provide a powerful prior for composing specialized models: each vector carries information about the expert’s data distribution, trajectory style, and learned behavior. However, agent task vectors cannot be composed reliably from weight-space geometry alone. Tool use, memory update, and code generation succeed or fail only after executable rollouts, and the coefficient that preserves one agent behavior may suppress another. We propose EXPERTGYM, a framework that turns frozen agent task-vector merging into a small, on-policy, executable-feedback-driven coefficient learning problem. Starting from a symmetric one-third prior over TOOL, MEMORY, and CODE experts, EXPERTGYM learns only global, layer-wise, or module-wise coefficients over frozen deltas. Calibration prompts are treated as probes rather than training examples: frontier prompts provide direction credit through group-relative optimization, same-prompt expert-recoverable failures provide recovery credit through verifier-grounded distillation, and stable successes provide boundary credit through retention. This framing distinguishes EXPERTGYM from both geometric merging and imitation-based expert alignment: expert trajectories are not universal supervision, but recovery evidence that must be validated by executable rewards and non-regression guards. We give a complete experimental framework for testing this claim, including state-distribution diagnostics, routed-versus-unrouted objectives, recovery-versus-imitation comparisons, coefficient granularity studies, and mixed-agent generalization.

1 Introduction

Specialized LLM agents are increasingly trained for different executable capabilities: one agent is optimized for function calling, another for memory update, another for code generation, and another

for reasoning or verification. Serving a router over all experts is expensive, while fully retraining a generalist agent is slow and may erase expert-specific behavior. Model merging offers an appealing middle ground: task vectors can be combined to produce a single deployed model without joint training or multi-expert inference (Wortsman et al., 2022; Ilharco et al., 2023).

The usual model-merging story is geometric. Given a base model and multiple experts, the method estimates how to add, trim, mask, align, or rescale task vectors so that parameter-side interference is reduced (Yadav et al., 2023; Yu et al., 2023; Deep et al., 2024; Gargiulo et al., 2025; Cheng et al., 2025). This view is valuable, but it leaves out the central property of agents: success is only revealed by executing a trajectory. A function-call model can have plausible logits while producing a non-parseable call; a memory model can answer a final question while corrupting intermediate state; a code model can imitate an expert trace but still fail unit tests. Thus the right coefficient for composing agent task vectors cannot be inferred from weight-space geometry alone.

This paper studies the following question: *How can we compose multiple agent task vectors when the right coefficient must be learned from executable trajectory feedback?* Our answer is EXPERTGYM, a *coefficient-space gym* for frozen agent task vectors. We begin from a symmetric one-third prior over TOOL, MEMORY, and CODE experts. This start is not a naive accident: each task vector is a structured capability prior learned from an expert’s data and trajectory distribution. The goal is not to relearn the experts from scratch, but to use a small calibration panel to redistribute probability mass among existing expert directions.

A key distinction is that calibration examples are not ordinary supervised training data. Because only coefficients are trainable while the base model and expert deltas remain frozen, calibration cannot

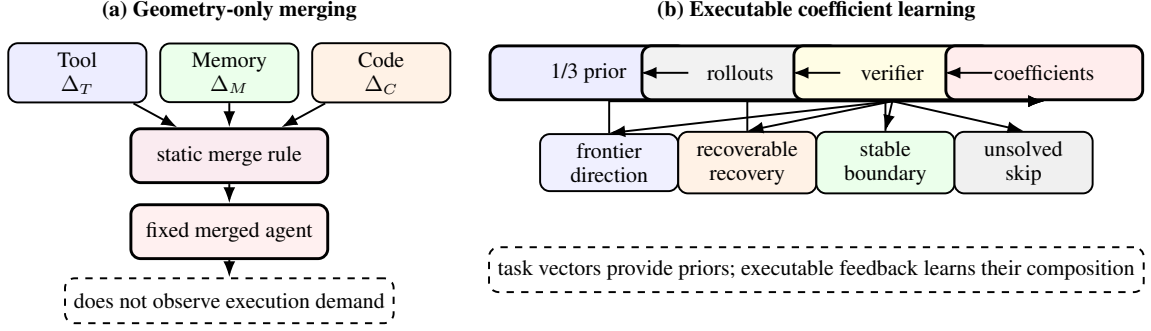


Figure 1: Central thesis. Static merging can combine task vectors using weight-space structure, but agent composition additionally requires rollout-level information: which trajectories succeed, which expert can recover a failure, and which updates break already executable behavior. EXPERTGYM keeps the task-vector prior frozen and learns only coefficients from executable feedback.

freely memorize outputs. Instead, prompts act as probes that reveal what kind of credit is available in the current merged policy. Frontier prompts, where some rollouts succeed and others fail, provide direction credit for group-relative optimization. Recoverable prompts, where the merge fails but a same-prompt expert succeeds, provide recovery credit. Stable prompts, where the merge already succeeds, define a non-regression boundary. Unsolved prompts, where neither merge nor experts succeed, should not update coefficients.

This framing also clarifies the relation to Expert Merging and imitation-based alignment. Expert Merging learns layer-wise coefficients by aligning hidden states and logits of a merged model to expert behavior on unlabeled calibration data (Zhang et al., 2025). Such alignment is useful, but it is not the same as executable success feedback. In EXPERTGYM, expert trajectories are not universal targets. They are used only when the current merged policy fails on the same prompt and an expert succeeds; the resulting recovery update is accepted only if executable rewards and retention guards show no cross-skill collapse. This turns distillation from imitation into verifier-grounded recovery.

Contributions. This version of the paper makes four claims that are experimentally testable with the current code framework.

- We formulate **agent task-vector composition** as low-dimensional coefficient learning over frozen expert deltas from executable feedback.
- We identify **Executable Credit Collapse**: many agent calibration prompts are all-success, all-fail, or expert-recoverable, so they do not supply the

same kind of coefficient credit.

- We propose **state-conditioned credit operators**: GRPO for frontier direction, same-prompt recovery distillation for suppressed expert priors, and retention for non-regression boundaries.
- We define a compact experimental program that separates geometry, imitation, recovery, and executable feedback, enabling rich diagnostics even before claiming state-of-the-art performance.

2 Background and Positioning

Task-vector priors. Let θ_0 be a base model and θ_e be an expert model for skill $e \in \mathcal{E}$. A task vector is $\Delta_e = \theta_e - \theta_0$. For agents, Δ_e carries more than a label-specific preference: it encodes the expert’s training data, decoding style, tool schema, memory protocol, and execution distribution. This is why task-vector composition is attractive. It preserves structure that would be costly to rediscover with full-parameter RL on a small calibration set.

Geometric merging. Static methods such as task arithmetic, TIES, DARE, DELLA, TSV, and WUDI improve how vectors are combined by handling sign conflict, redundancy, sparsity, singular structure, or interference (Ilharco et al., 2023; Yadav et al., 2023; Yu et al., 2023; Deep et al., 2024; Gargiulo et al., 2025; Cheng et al., 2025). These methods largely do not learn from the current merged agent’s executable trajectories. Our position is not that geometry is wrong; rather, geometry supplies a prior, but agent feedback supplies the missing demand signal.

Agent merging. Recent agent-oriented work argues that agent deltas are sparse, heterogeneous,

and role-sensitive. RAM studies reinforced agentic model merging and emphasizes behavior-knowledge preservation in RL-trained agents (Yuan et al., 2026). ARM proposes role-conditioned neuron transplantation for training-free generalist agent merging (Feng et al., 2026). These works motivate the same community problem: agent merging is not just static task merging. EXPERTGYM is complementary because it focuses on verifier-side coefficient learning: how should a frozen composition move once we can execute the merged agent?

Imitation and on-policy distillation. On-policy distillation reduces mismatch by learning around outputs generated by the student policy rather than only teacher-forced data (Agarwal et al., 2024). But in our setting, pure imitation is not enough. Hidden-state or logit matching can align the merged model to experts without knowing whether the current merge actually failed, whether the expert was needed, or whether the update harms another agent behavior. EXPERTGYM therefore treats distillation as a state-triggered recovery operator, not a general training objective.

3 Problem Formulation

3.1 Coefficient Spaces

The global composition is

$$\theta(\alpha) = \theta_0 + \sum_{e \in \mathcal{E}} \alpha_e \Delta_e. \quad (1)$$

For three experts $\mathcal{E} = \{T, M, C\}$, the default initialization is

$$\theta_{1/3} = \theta_0 + \frac{1}{3}(\Delta_T + \Delta_M + \Delta_C). \quad (2)$$

A stable parameterization is common-plus-residual:

$$\alpha_e = c + r_e, \quad (3)$$

where c controls shared expert strength and r_e controls expert-specific residuals. This makes training a controlled deviation from a symmetric prior rather than unconstrained coefficient search.

For fine-grained control, split parameters into groups $g \in \mathcal{G}$ such as layer bands or transformer modules:

$$\theta(\alpha) = \theta_0 + \sum_{e \in \mathcal{E}} \sum_{g \in \mathcal{G}} \alpha_{e,g} \Delta_{e,g}. \quad (4)$$

The module-level setting supports micro-function control, e.g., attention routing versus FFN transformation. It should be presented as a capacity study,

not as the default main claim, unless experiments show clear non-regressive gains.

3.2 Executable Feedback

Given prompt x , the current merged policy samples K trajectories

$$y_1, \dots, y_K \sim \pi_{\theta(\alpha)}(\cdot|x), \quad r_i = V(x, y_i), \quad (5)$$

where V is a task verifier. TOOL rewards parseable function calls with correct arguments, MEMORY rewards stateful memory update and final-answer correctness, and CODE rewards code execution under official or generated tests. These rewards are deliberately aligned with the original expert training environments when possible.

Define the empirical success rate $\hat{p}(x)$ and expert recoverability

$$U(x) = \mathbb{I}[\exists e, \exists y_e^+ \sim \pi_{\theta_e}(\cdot|x) : V(x, y_e^+) > 0]. \quad (6)$$

A prompt state is then assigned as follows:

$$S(x) = \begin{cases} \text{FRONTIER}, & 0 < \hat{p}(x) < 1, \\ \text{RECOVERABLE}, & \hat{p}(x) = 0 \text{ and } U(x) = 1, \\ \text{STABLE}, & \hat{p}(x) \approx 1, \\ \text{UNSOLVED}, & \hat{p}(x) = 0 \text{ and } U(x) = 0. \end{cases} \quad (7)$$

Executable Credit Collapse. If most prompts are all-fail or all-success, then direct policy optimization over coefficients is compute-heavy but signal-poor. If recoverable prompts are treated as ordinary imitation data, coefficients may drift toward one expert while hurting others. If stable prompts are ignored, an apparent improvement can erase tool formats, memory state updates, or code executability. We call this family of failures *Executable Credit Collapse*.

4 Method: ExpertGym

EXPERTGYM constructs a small on-policy calibration environment for coefficient learning. It does not update base or expert weights. It repeatedly samples rollouts from the current merged policy, verifies outcomes, routes prompts by state, applies the valid credit operator, and accepts only non-regressive coefficient updates.

4.1 Frontier Direction: GRPO over Coefficients

For frontier prompts, rewards vary among sampled trajectories. We use a group-relative objective

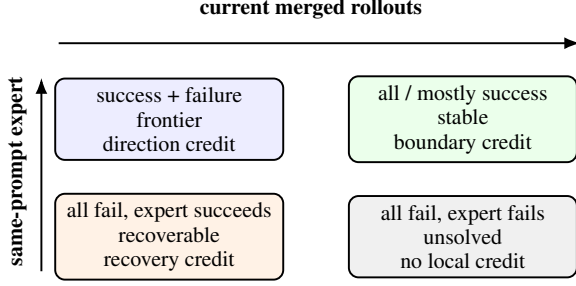


Figure 2: Execution-state taxonomy. The same calibration prompt can play different roles depending on current policy rollouts and same-prompt expert recovery. This taxonomy is the bridge between theory and method.

inspired by GRPO (Shao et al., 2024). With normalized advantage

$$\hat{A}_i = \frac{r_i - \bar{r}}{\text{std}(r_1, \dots, r_K) + \epsilon}, \quad (8)$$

we update coefficients with a clipped objective

$$\mathcal{L}_G = -\mathbb{E} \left[\min \left(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1 - \epsilon_c, 1 + \epsilon_c) \hat{A}_i \right) \right], \quad (9)$$

where ρ_i is computed between current and old coefficients. This term is only valid where reward variance exists. The paper should not claim that GRPO solves all coefficient learning; it supplies direction credit on the frontier subset.

4.2 Recoverable Failures: Distillation as Verifier-Grounded Recovery

For recoverable prompts, current rollouts fail but a same-prompt expert succeeds. Let y_e^+ denote that successful expert trajectory. The recovery loss is

$$\mathcal{L}_D = -\mathbb{E}_{(x,e,y_e^+)} w_{x,e} \frac{1}{|y_e^+|^\gamma} \log \pi_{\theta(\alpha)}(y_e^+ | x). \quad (10)$$

This is deliberately not generic imitation. The trigger requires current failure and expert success on the same prompt. The weight $w_{x,e}$ can incorporate expert margin, task balance, and gradient agreement with frontier updates. Candidate updates induced by $\mathcal{L}_D$ must pass executable validation before being kept.

Why this is different from Expert Merging. Expert Merging aligns hidden states and logits on unlabeled calibration data to learn layer-wise coefficients (Zhang et al., 2025). Our recovery objective is narrower and more agentic: it asks whether an expert trajectory recovers a concrete failure of the

State	Observable condition	Credit	Operator
Frontier	mixed rollout rewards	direction	GRPO
Recoverable	merge fails, expert succeeds	recovery	same-prompt OPD
Stable	merge already succeeds	boundary	retention
Unsolved	merge and experts fail	none	skip / data acquisition

Table 1: Core method as an evidence-to-objective map. This table should remain central in the final paper: it makes the loss design look inevitable rather than heuristic.

current merge, and whether the resulting coefficient movement improves executable outcomes without breaking other skills. This distinction should be made explicit in the experiments through imitation-only and hidden/logit-alignment baselines.

4.3 Stable Successes: Retention as Boundary Credit

Stable prompts define safe regions. Let y^+ be a successful response from the reference or accepted merged policy. Retention uses

$$\mathcal{L}_R = -\mathbb{E}_{(x,y^+) \in \mathcal{G}} \log \pi_{\theta(\alpha)}(y^+ | x). \quad (11)$$

This term is not expected to discover new capability. Its role is to mark a boundary: updates should not erase parseability, memory update consistency, or code execution patterns that are already solved. This framing makes retention a necessary guard rather than an arbitrary regularizer.

4.4 Credit Normalization and Guarded Update

The total objective is

$$\mathcal{L} = \lambda_G \mathcal{L}_G + \lambda_D \mathcal{L}_D + \lambda_R \mathcal{L}_R + \lambda_2 \|\alpha - \alpha_{1/3}\|_2^2. \quad (12)$$

Because recoverable prompts can be numerous early in training, we recommend gradient-budget balancing:

$$\lambda_j \leftarrow \text{clip} \left(\frac{b_j}{\text{EMA}(\|\nabla_{\alpha} \mathcal{L}_j\|) + \epsilon}, \lambda_j^{\min}, \lambda_j^{\max} \right), \quad (13)$$

for $j \in \{G, D, R\}$. After each interval, a candidate coefficient update is evaluated on a held-out guard set. It is accepted only if average reward improves or remains stable and worst-task regression is below a threshold δ .

5 Experiments

The experimental program is designed to support the story before chasing leaderboard claims. The

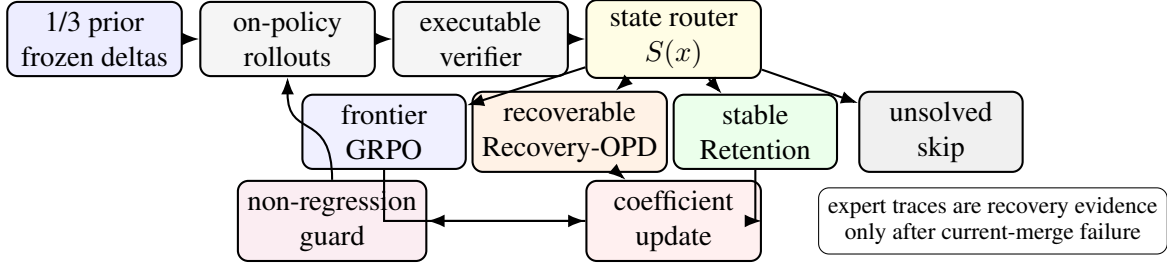


Figure 3: EXPERTGYM training loop. The algorithm is a coefficient-space agent gym: a small calibration set creates real rollouts, verifiers assign execution states, and each state selects the valid credit operator.

Model	Tool	Memory	Code	Avg.	Worst drop
Base	TBD	TBD	TBD	TBD	–
Tool expert	TBD	TBD	TBD	TBD	TBD
Memory expert	TBD	TBD	TBD	TBD	TBD
Code expert	TBD	TBD	TBD	TBD	TBD
Equal-prior $\theta_{1/3}$	TBD	TBD	TBD	TBD	ref.

Table 2: Task-vector prior table. This table should show that expert deltas contain useful skill priors, while equal-prior composition still leaves room for feedback-conditioned coefficient learning.

key is to show that EXPERTGYM solves a problem that geometric merging and imitation-only alignment do not address.

5.1 Benchmarks and Metrics

TOOL uses function-call exactness, argument correctness, parseability, and schema validity. MEMORY uses multi-turn memory update with final-answer EM/F1. CODE uses Pass@1, any-pass@K reachability, and best-of- K selection accuracy. Main tables should report average score, worst-task drop relative to $\theta_{1/3}$, and collapse count. A method that increases average while breaking one skill should not be considered successful.

5.2 E1: Task Vectors Contain Structured Priors

This experiment establishes why the one-third prior is meaningful. It compares base, single experts, and $\theta_{1/3}$. The expected conclusion is not that $\theta_{1/3}$ is optimal, but that it is stronger than base and exposes real multi-skill behavior while still showing composition trade-offs.

5.3 E2: Geometry Cannot Infer Executable Demand

This experiment compares static geometric merging to executable-feedback-conditioned learning. Baselines should include task arithmetic, best static scale sweep, TIES, DARE+TA, WUDI/TSV-style

structure-aware merging, RAM/ARM-style agent merging if available, and Expert Merging-style coefficient alignment. The current filled numbers report Tool mean, Memory F1, and Code Pass@1/Acc. under the same Eval6 harness; average is the mean of those three columns.

5.4 E3: Calibration State Distribution

This table is the empirical basis for the paper’s insight. We roll out $\theta_{1/3}$ on 96 balanced evaluation-targeted calibration prompts with $K = 8$ samples and classify each prompt by current rollout outcomes and same-prompt expert recoverability.

5.5 E4: Routing Matters More than Loss Mixing

5.6 E5: Recovery Is Not Imitation

This is the most important defensive experiment. It should compare generic expert imitation, Expert Merging-style hidden/logit alignment, offline expert traces, same-prompt recoverable OPD, and verifier-guarded recovery.

5.7 E6: Code Planning Diagnostic

Because code success depends on planning and execution, imitation-only metrics are insufficient. Split code into reachability and selection.

5.8 E7: Coefficient Granularity and Overfit

The module-level story is attractive but must be handled carefully. The right claim is that finer coefficient spaces provide more local repair directions, but only if retention and held-out validation prevent overfitting.

5.9 E8: Mixed-Agent Generalization

To make the paper more than a benchmark-specific merge, add a small mixed-agent evaluation set: Tool+Memory, Memory+Code, Tool+Code, and Tool+Memory+Code prompts. These prompts test

Method	Learns?	Exec. feedback?	Tool	Memory	Code	Avg.	Worst drop	Collapse
TA 1/3	no	no	0.785	0.646	0.341	0.591	ref.	0
TA 0.75	no	no	0.785	0.759	0.349	0.631	0.000	0
TIES	no	no	0.764	0.636	0.336	0.579	-0.021	2
DARE+TA	no	no	0.795	0.690	0.337	0.607	-0.004	0
DARE+TIES	no	no	0.795	0.689	0.343	0.609	0.002	0
AdaMerging	coeff.	entropy	0.784	0.668	0.341	0.597	-0.001	0
Mixture GRPO	full	yes	0.782	0.664	0.338	0.595	-0.003	0
EXPERTGYM global 3	coeff.	yes	0.792	0.666	0.342	0.600	0.001	0
EXPERTGYM common+res.	coeff.	yes	0.786	0.690	0.337	0.604	-0.004	0

Table 3: Main comparison snapshot. Static scale selection is a strong baseline on Memory, while executable-feedback coefficient learning gives non-regressive movement from the equal-prior start. The current numbers support a cautious claim about learnable composition rather than a SOTA claim.

Skill	Frontier	Recover.	Stable	Unsolved	Std.
Tool	14	1	10	7	0.243
Memory	28	3	0	1	0.389
Code	12	6	2	12	0.197
All	54	10	12	20	0.276

Table 4: Executable state distribution at the equal-prior start. Frontier prompts justify GRPO, recoverable prompts justify same-prompt recovery-OPD, and stable prompts justify retention. Code has the largest unsolved mass, which explains why Code improvement remains the hardest held-out dimension.

Method	GRPO	OPD	Ret.	Routed	Avg.	Drop
GRPO only	yes	no	no	partial	TBD	TBD
OPD only	no	yes	no	partial	TBD	TBD
Retention only	no	no	yes	partial	TBD	TBD
Unrouted weighted sum	yes	yes	yes	no	TBD	TBD
Random routing	yes	yes	yes	random	TBD	TBD
EXPERTGYM	yes	yes	yes	state	TBD	TBD

Table 5: Loss and routing ablation. The crucial result is EXPERTGYM beating the unrouted weighted sum, which would show that the contribution is execution-state credit assignment rather than a tuned loss mixture.

whether task vectors can produce combination generalization when executable feedback learns the composition.

6 Analysis Figures to Fill

Expected useful findings. Even if final scores are modest, the paper can be strong if it shows: (i) frontier signal is sparse, explaining why pure GRPO is slow; (ii) recoverable prompts exist, explaining why recovery is needed; (iii) generic imitation creates cross-task regression, while same-prompt recovery plus guard is safer; and (iv) coefficient updates reveal interpretable skill trade-offs.

7 Possible Solutions for Current Bottlenecks

If OPD dominates coefficient movement. Do not hide it. Reframe OPD as early-phase recovery and report its gradient share. Add a schedule that decays recovery weight as frontier prompts increase. A useful diagnostic is the percentage of recovered prompts that later become frontier or stable under the updated merge.

If GRPO is weak. Restrict GRPO claims to frontier prompts. Improve its signal by frontier oversampling, reward shaping from verifier components, and per-task advantage normalization. For code, reward shaping can separate parse success, compilation, unit-test pass, and hidden-test pass.

If retention only prevents collapse. That is the intended role. Present retention sweeps as a Pareto boundary: higher retention should reduce worst-task drop while potentially limiting gains. The figure should show a trade-off curve, not a single magic coefficient.

If module gates overfit. Use held-out calibration guards and coefficient sparsity. Module-level results can be framed as analysis unless they clearly improve non-regressive metrics.

If Expert Merging is a strong baseline. That is useful. Position EXPERTGYM as adding executable recovery and non-regression guards on top of coefficient learning. The most important direct comparison is hidden/logit alignment versus same-prompt recovery under executable metrics.

8 Related Work

Model merging and task vectors. Model soups showed that weight averaging can improve accuracy without additional inference cost (Wortsman

Method	Trigger	Signal	Verifier guard	Tool	Memory	Code	Avg.	Drop
Offline trace distill	always	expert traj.	no	TBD	TBD	TBD	TBD	TBD
Logit imitation	always	expert logits	no	TBD	TBD	TBD	TBD	TBD
Hidden/logit align	always	expert states	no	TBD	TBD	TBD	TBD	TBD
Same-prompt OPD	merge all-fail	expert success	no	TBD	TBD	TBD	TBD	TBD
Same-prompt OPD + guard	merge all-fail	expert success	yes	TBD	TBD	TBD	TBD	TBD
Full EXPERTGYM	state-conditioned	mixed credit	yes	TBD	TBD	TBD	TBD	TBD

Table 6: Recovery versus imitation. This experiment directly addresses the risk that distillation looks like ordinary imitation. The target result is not necessarily that recovery always gives the largest raw gain, but that it gives safer executable improvement under worst-task non-regression.

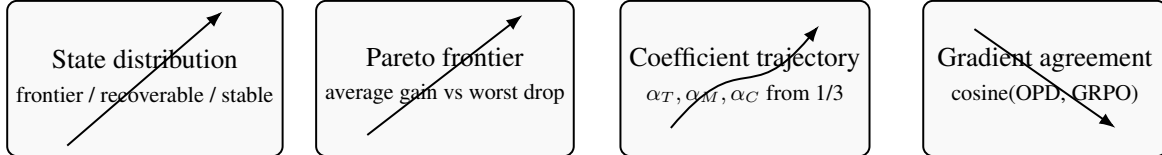


Figure 4: Analysis dashboard template. These four plots make the paper visually rich without overclaiming: they show where signal comes from, how non-regression is enforced, how coefficients move, and whether recovery and direction gradients conflict.

Method	Any-pass@K	BoN select	Pass@1	Main failure
$\theta_{1/3}$	TBD	TBD	TBD	TBD
OPD only	TBD	TBD	TBD	TBD
GRPO only	TBD	TBD	TBD	TBD
Full EXPERTGYM	TBD	TBD	TBD	TBD

Table 7: Code diagnostic. Recovery may improve reachability by moving the merge toward expert-like correct trajectories, while executable feedback is needed to validate planning-sensitive outcomes.

Space	Params	Tool	Mem.	Code	Avg.	Gap
Global 3	3	TBD	TBD	TBD	TBD	TBD
Common+residual	4	TBD	TBD	TBD	TBD	TBD
Layer-wise	TBD	TBD	TBD	TBD	TBD	TBD
Module-wise	588	TBD	TBD	TBD	TBD	TBD
Module + guard	588	TBD	TBD	TBD	TBD	TBD

Table 8: Coefficient capacity ladder. If global coefficients already capture most gains, the paper can argue that task-vector priors are strong. If module gates improve results, the paper can argue for localized micro-function repair.

et al., 2022). Task arithmetic framed fine-tuning deltas as editable capability vectors (Ilharco et al., 2023). TIES, DARE, DELLA, TSV, and WUDI reduce interference through sign resolution, sparsification, sampling, singular structure, or data-free vector guidance (Yadav et al., 2023; Yu et al., 2023; Deep et al., 2024; Gargiulo et al., 2025; Cheng et al., 2025). EXPERTGYM keeps this prior but learns how to activate it from executable rollouts.

Agent merging. RAM and ARM extend merging concerns to reinforced or interactive agents,

Method	T+M	M+C	T+C	T+M+C	Avg.
$\theta_{1/3}$	TBD	TBD	TBD	TBD	TBD
Static best	TBD	TBD	TBD	TBD	TBD
Imitation-only	TBD	TBD	TBD	TBD	TBD
Full EXPERTGYM	TBD	TBD	TBD	TBD	TBD

Table 9: Mixed-agent generalization. This is the most attractive optional experiment: it tests whether coefficient learning unlocks behavior combinations not directly represented by any single expert.

emphasizing sparse behavior knowledge, role-conditioned activations, and training-free integration (Yuan et al., 2026; Feng et al., 2026). EXPERTGYM is complementary: instead of deciding which neurons to transplant, it asks how frozen agent vectors should be scaled after observing actual execution outcomes.

Expert alignment and distillation. Expert Merging learns layer-wise coefficients using hidden-state and logit alignment on unlabeled calibration data (Zhang et al., 2025). On-policy distillation addresses distribution mismatch by learning from student-generated behavior (Agarwal et al., 2024). We use expert behavior more narrowly: same-prompt expert success is recovery evidence for a current merge failure, not universal supervision.

Executable feedback. Agent benchmarks such as BFCL, memory-grounded QA, LiveCodeBench, and code-unit-test reinforcement show why final

executable reward differs from static next-token similarity (Patil et al., 2025; Yang et al., 2018; Jain et al., 2025; Wang et al., 2025). EXPERTGYM uses this property as the core learning signal for coefficient composition.

9 Conclusion

EXPERTGYM reframes agent merging as executable-feedback-driven coefficient learning. Task vectors provide structured priors; executable trajectories tell us how to compose them. The paper’s main novelty is not a larger coefficient space or a stronger imitation loss, but a state-conditioned calibration framework that separates direction, recovery, and boundary credit. This gives a plausible and defensible path from current experiments to a review-ready EMNLP paper: show that geometry alone misses rollout demand, imitation alone lacks success-grounded credit, and small coefficient-space learning can improve or stabilize agent task-vector composition under non-regression constraints.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*.
- Runxi Cheng, Feng Xiong, Yongxian Wei, Wanyun Zhu, and Chun Yuan. 2025. Whoever started the interference should end it: Guiding data-free model merging via task vectors. In *International Conference on Machine Learning*.
- Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. 2024. DELLA-merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*.
- Zhuoka Feng, Kang Chen, Sihan Zhao, Kai Xiong, Yaoning Wang, Minshen Yu, Junjie Nian, Changyi Xiao, Yixin Cao, and Yugang Jiang. 2026. ARM: Role-conditioned neuron transplantation for training-free generalist LLM agent merging. *arXiv preprint arXiv:2601.07309*.
- Antonio Andrea Gargiulo, Donato Crisostomi, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, and Emanuele Rodola. 2025. Task singular vectors: Reducing task interference in model merging. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hananeh Hajishirzi, and Ali Farhadi. 2023. Editing models with task arithmetic. In *International Conference on Learning Representations*.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fan-jia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Live-codebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2025. Berkeley function calling leaderboard: A challenging and executable function calling benchmark. In *Proceedings of Machine Learning Research*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Yibo Wang and 1 others. 2025. Co-evolving LLM coder and unit tester via reinforcement learning. *arXiv preprint arXiv:2506.03136*.
- Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*, pages 23965–23998.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. 2023. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380.
- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. 2023. Language models are super mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*.
- Xiangchi Yuan, Dachuan Shi, Chunhui Zhang, Zheyuan Liu, Shenglong Yao, Soroush Vosoughi, and Wenke Lee. 2026. Behavior knowledge merge in reinforced agentic models. *arXiv preprint arXiv:2601.13572*.

Dengming Zhang, Xiaowen Ma, Zhenliang Ni, Zhenkai Wu, Han Shu, Xin Jiang, and Xinghao Chen. 2025. Expert merging: Model merging with unsupervised expert alignment and importance-guided layer chunking. *arXiv preprint arXiv:2509.25712*.

A Agent Group Synthesis

Strategist. The paper must sell a problem, not a training recipe. The center sentence is: task vectors provide the prior; executable feedback learns the composition.

Method architect. Distillation is the weak point if presented as imitation. It becomes novel only when triggered by current-merge failure, same-prompt expert success, and verifier acceptance.

Experiment designer. The first result should be state distribution, not final benchmark score. The key ablation is state-routed objective versus unrouted loss mixture.

Skeptical reviewer. Do not claim SOTA before RAM, ARM, Expert Merging, and geometric baselines are implemented on the same suite. The safer claim is non-regressive improvement and diagnostic insight.

B Minimum Run Plan

1. Roll out $\theta_{1/3}$ on 100–150 balanced calibration prompts with $K = 8$ or $K = 16$ samples. Fill Table 4.
2. Run global 3 and common+residual 4 coefficient spaces before module-level gates.
3. Compare GRPO-only, OPD-only, retention-only, full routed EXPERTGYM, unrouted sum, random routing, and task-only routing.
4. Compare offline trace distillation, logit imitation, hidden/logit Expert-Merging-style alignment, same-prompt OPD, and verifier-guarded recovery.
5. For code, report any-pass@K, BoN selection, and Pass@1 rather than only aggregate accuracy.
6. Add mixed-agent prompts after the single-skill loop works. This is the highest-impact optional experiment.

C Claim Ladder

Safe claim. EXPERTGYM formulates agent task-vector composition as executable-feedback-driven coefficient learning and provides diagnostics that static geometric merging lacks.

Medium claim. State routing is necessary because frontier, recoverable, and stable prompts provide different valid credit types.

Strong claim. EXPERTGYM improves over geometric and imitation-only merging under non-regression across Tool, Memory, and Code.

Do not claim yet. State-of-the-art agent merging, unless all strong baselines are run on the same held-out suite.